

გაკვეთილი 3 Mod 4 (4.2)

გაკვეთილი 1 — TypeScript-ის შესავალი, ცვლადები და ტიპების მითითება

გაკვეთილის მიზანი

გაკვეთილის ბოლოს მოსწავლემ უნდა შეძლოს:

- ახსნას, რა არის TypeScript;
- ახსნას JavaScript-სა და TypeScript-ს შორის მთავარი განსხვავება;
- გამოიყენოს უკვე ნაცნობი `let` და `const`;
- ცვლადს ხელით მიუთითოს ტიპი;
- გაიგოს, რატომ გვიცავს TypeScript არასწორი ტიპის მნიშვნელობის მინიჭებისგან;
- გაიგოს TypeScript-ის შეცდომის მარტივი შეტყობინება;
- გამოიყენოს `number`, `string` და `boolean` საწყის მაგალითებში.

1. გავიხსენოთ JavaScript

დაიწყე უკვე ნაცნობი კოდით:

```
let name = "Giorgi";  
let age = 20;  
let isStudent = true;
```

ჰკითხე ბავშვებს:

რა არის `name`, `age` და `isStudent`?

უკვე უნდა იცოდნენ, რომ ეს ცვლადებია.

შემდეგ:

```
let score = 10;  
score = 20;
```

ესეც ნაცნობია — `let`-ით შექმნილი ცვლადის მნიშვნელობა შეიძლება შეიცვალოს.

ასევე:

```
const country = "Georgia";
```

`const` -ს ახალ მნიშვნელობას ვეღარ მივანიჭებთ.

აქ დიდხანს აღარ გავჩერდებით, რადგან ეს JavaScript-იდან უკვე იციან.

2. მაშინ რატომ გვჭირდება TypeScript?

აქ იწყება ახალი მასალა.

JavaScript-ში შეგვიძლია დავწეროთ:

```
let age = 20;  
  
age = "twenty";
```

JavaScript ამ კოდის დაწერას გვაძლევს.

თავიდან `age` -ში გვქონდა:

```
20 → რიცხვი
```

შემდეგ:

```
"twenty" → ტექსტი
```

დიდ პროგრამაში ასეთმა ცვლილებამ შეიძლება მოგვიანებით პრობლემა გამოიწვიოს.

მაგალითად:

```
let price = 100;  
  
price = "hello";
```

ჩვენ გვინდოდა `price` ყოფილიყო პროდუქტის ფასი, მაგრამ შემთხვევით ტექსტი შევინახეთ.

TypeScript-ის ერთ-ერთი მთავარი დანიშნულება სწორედ ასეთი შეცდომების ადრე აღმოჩენაა.

3. TypeScript რა არის?

TypeScript არის JavaScript-ზე დაფუძნებული პროგრამირების ენა.

ძალიან მარტივად:

TypeScript = JavaScript + ტიპების კონტროლი

JavaScript-ის ცოდნა არ იკარგება. პირიქით, TypeScript-ში ისევ ვიყენებთ:

```
let  
const  
if  
else  
for  
while  
function
```

და JavaScript-ის სხვა ნაცნობ კონსტრუქციებს.

მაგრამ TypeScript დამატებით გვაძლევს შესაძლებლობას, პროგრამას ვუთხრათ:

„ამ ცვლადში მხოლოდ რიცხვი უნდა იყოს.“

ან:

„ამ ცვლადში მხოლოდ ტექსტი უნდა იყოს.“

4. Type Annotation — ტიპის პირდაპირ მითითება

ეს არის პირველი მნიშვნელოვანი ახალი რამ.

JavaScript-ში გვქონდა:

```
let age = 20;
```

TypeScript-ში შეგვიძლია დავწეროთ:

```
let age: number = 20;
```

ახალი ნაწილი არის:

```
: number
```

ამას ეწოდება **Type Annotation** — ტიპის პირდაპირ მითითება.

სტრუქტურა ასეთია:

```
let age: number = 20;
```

დავშალოთ:

let	age	:	number	=	20
	↓		↓		↓
	სახელი		ტიპი		მნიშვნელობა

ანუ პროგრამას ვეუბნებით:

შექმენი `age` ცვლადი, რომელიც ინახავს `number` ტიპის მნიშვნელობას.

5. რატომ არის ეს მნიშვნელოვანი?

დავწეროთ:

```
let age: number = 20;
```

შემდეგ:

```
age = 21;
```

ყველაფერი სწორია.

რადგან:

```
number → number
```

მაგრამ თუ დავწერთ:

```
age = "Giorgi";
```

TypeScript შეცდომას გვაჩვენებს.

რატომ?

იმიტომ, რომ ჩვენ თავიდანვე ვთქვით:

```
age: number
```

ანუ `age` უნდა იყოს რიცხვი.

მაგრამ:

```
"Giorgi"
```

არის ტექსტი.

6. JavaScript vs TypeScript

ეს მაგალითი დაფაზე აუცილებლად აჩვენე.

JavaScript

```
let score = 100;  
  
score = "hello";
```

JavaScript-ში ცვლადის ტიპი შეიძლება შეიცვალოს.

TypeScript

```
let score: number = 100;  
  
score = "hello";
```

TypeScript გვაჩვენებს შეცდომას.

სწორი იქნება:

```
score = 200;
```

ამიტომ TypeScript-ის ერთ-ერთი უპირატესობაა:

შეცდომას ხშირად კოდის წერის დროსვე ვხედავთ და არა მხოლოდ პროგრამის გაშვების შემდეგ.

7. სამი ძირითადი ტიპი

ტიპებს შემდეგ გაკვეთილზე უფრო ღრმად გაივლიან, მაგრამ პირველი გაკვეთილიდან უნდა მიეჩვიონ TypeScript-ის სინტაქსს.

number

რიცხვებისთვის:

```
let age: number = 16;  
let score: number = 100;  
let price: number = 25.5;
```

მთელი და ათწილადი რიცხვებიც `number` არის.

string

ტექსტისთვის:

```
let name: string = "Nika";  
let city: string = "Khoni";  
let country: string = "Georgia";
```

მაგალითად:

```
let name: string = "Nika";  
  
name = "Ana";
```

სწორია.

მაგრამ:

```
name = 100;
```

არასწორია, რადგან `name` არის `string`.

boolean

`boolean`-ს მხოლოდ ორი შესაძლო მნიშვნელობა აქვს:

```
true  
false
```

მაგალითად:

```
let isStudent: boolean = true;
```

ან:

```
let isOnline: boolean = false;
```

შეგვიძლია შევცვალოთ:

```
isOnline = true;
```

მაგრამ:

```
isOnline = "yes";
```

არასწორია.

8. `let` და `const` TypeScript-ში

აქ უკვე ნაცნობ ცოდნას TypeScript-ს ვუკავშირებთ.

```
let score: number = 0;
```

`score` შეიძლება შეიცვალოს:

```
score = 10;
```

```
score = 20;
```

ხოლო:

```
const schoolName: string = "Khoni School";
```

`schoolName` -ს ხელახლა მნიშვნელობას ვერ მივანიჭებთ.

მაგალითად ეს არასწორია:

```
schoolName = "Other School";
```

აქ მნიშვნელოვანია ბავშვებმა ორი სხვადასხვა იდეა არ აურიონ:

`let` / `const` გვეუბნება, შეიძლება თუ არა მნიშვნელობის ხელახლა მინიჭება.

ხოლო:

`number` / `string` / `boolean` გვეუბნება, რა ტიპის მონაცემს ვინახავთ.

მაგალითად:

```
let score: number = 10;
```

`let` → მნიშვნელობა შეიძლება შეიცვალოს.

`number` → მაგრამ ახალი მნიშვნელობაც რიცხვი უნდა იყოს.

ამიტომ:

```
score = 50;
```

✓ სწორია.

```
score = "fifty";
```

✗ არასწორია.

9. TypeScript-ის კიდევ ერთი საინტერესო შესაძლებლობა — Type Inference

ახლა აჩვენე:

```
let age = 20;
```

აქ ჩვენ არ დაგვიწერია:

```
: number
```

მაგრამ TypeScript თვითონ ხვდება, რომ 20 რიცხვია.

ამას ეწოდება:

Type Inference

ანუ **TypeScript** თვითონ განსაზღვრავს ტიპს მნიშვნელობის მიხედვით.

მაგალითად:

```
let name = "Giorgi";
```

TypeScript ხვდება:

```
name → string
```

```
let age = 20;
```

ხვდება:

```
age → number
```

```
let isStudent = true;
```

ხვდება:

```
isStudent → boolean
```

ამიტომ შემდეგ:

```
let age = 20;
```

```
age = "hello";
```

TypeScript მაინც შეცდომას გვაჩვენებს.

მიუხედავად იმისა, რომ ჩვენ ხელით არ დაგვიწერია:

```
: number
```

TypeScript-მა 20 -ის მიხედვით თვითონ დაასკვნა, რომ age არის number .

10. Type Annotation და Type Inference

ეს ორი ტერმინი აუცილებლად გაარჩიე.

Type Annotation

ჩვენ თვითონ ვუთითებთ ტიპს:

```
let age: number = 20;
```

Type Inference

TypeScript თვითონ ადგენს ტიპს:

```
let age = 20;
```

ორივე შემთხვევაში age არის number .

პირველ შემთხვევაში ჩვენ მივუთითეთ, მეორე შემთხვევაში **TypeScript** მიხვდა.

11. რატომ არ დავწეროთ ყველგან ტიპი?

მაგალითად:

```
let age: number = 20;
```

სრულიად სწორია.

მაგრამ TypeScript-ს შეუძლია თვითონაც მიხედეს:

```
let age = 20;
```

ამიტომ რეალურ კოდში ორივე ფორმას შეხვდებიან.

დასაწყისში კი სავარჯიშოებში ხშირად ხელით მივუთითოთ ტიპები, რათა მოსწავლეებმა კარგად გაიაზრონ:

```
let name: string = "Nika";  
let age: number = 16;  
let isStudent: boolean = true;
```

12. შეცდომების წაკითხვა

TypeScript-ის სწავლისას მნიშვნელოვანია, ბავშვებს წითელი ხაზის არ შეეშინდეთ.

მაგალითად:

```
let age: number = 16;  
  
age = "seventeen";
```

IDE-მ შეიძლება დაახლოებით ასეთი შეცდომა გვაჩვენოს:

```
Type 'string' is not assignable to type 'number'.
```

მარტივად ეს ნიშნავს:

„შენ ცდილობ ტექსტი ჩაწერო იქ, სადაც რიცხვს ველოდები.“

აქ შეგიძლია უთხრა ბავშვებს:

TypeScript-ის შეცდომა ჩვენი მტერია კი არა, გვებმარება ვიპოვოთ სად დავუშვით შეცდომა.

13. პრაქტიკული მაგალითი — მოსწავლე

JavaScript-ში შეიძლება გვექნოდეს:

```
let studentName = "Nika";  
let age = 16;  
let score = 90;  
let isStudent = true;
```

TypeScript-ში ტიპები პირდაპირ მივუთითოთ:

```
let studentName: string = "Nika";  
let age: number = 16;  
let score: number = 90;  
let isStudent: boolean = true;
```

შემდეგ:

```
score = 95;  
age = 17;
```

სწორია.

მაგრამ:

```
score = "Excellent";
```

არასწორია.

რადგან:

```
score: number
```

14. პრაქტიკული მაგალითი — თამაში

```
const gameName: string = "Robot Game";  
  
let playerName: string = "Giorgi";  
let score: number = 0;  
let lives: number = 3;  
let gameOver: boolean = false;
```

თამაშის დროს:

```
score = 50;  
lives = 2;
```

შემდეგ:

```
score = 100;  
lives = 1;
```

ბოლოს:

```
gameOver = true;
```

ეს კარგი მაგალითია იმის საჩვენებლად, რომ TypeScript-ში JavaScript-ის ნაცნობი ლოგიკა ისევ გვაქვს, უბრალოდ მონაცემების ტიპებს უფრო მკაცრად ვაკონტროლებთ.

პრაქტიკული სავარჯიშოები

სავარჯიშო 1 — მიუთითე ტიპები

მოცემულია JavaScript-ის კოდი:

```
let name = "Ana";  
let age = 16;  
let score = 85;  
let isStudent = true;
```

გადააკეთეთ TypeScript-ის კოდად და თითოეულ ცვლადს პირდაპირ მიუთითეთ ტიპი.

პასუხი

```
let name: string = "Ana";  
let age: number = 16;  
let score: number = 85;  
let isStudent: boolean = true;
```

სავარჯიშო 2 — იპოვე შეცდომები

```
let name: string = "Giorgi";
let age: number = 20;
let isStudent: boolean = true;

name = 25;
age = "twenty";
isStudent = "yes";
```

მოსწავლემ უნდა იპოვოს სამივე შეცდომა.

სწორი ვარიანტი შეიძლება იყოს:

```
name = "Nika";
age = 21;
isStudent = false;
```

სავარჯიშო 3 — რა ტიპია?

მიუწერეთ შესაბამისი ტიპი:

```
"Giorgi" → ?
25 → ?
true → ?
"Georgia" → ?
19.5 → ?
false → ?
```

პასუხები:

```
"Giorgi" → string
25 → number
true → boolean
"Georgia" → string
19.5 → number
false → boolean
```

სავარჯიშო 4 — Type Inference

მოცემულია:

```
let city = "Khoni";  
let temperature = 25;  
let isRaining = false;
```

უპასუხონ:

1. რა ტიპი მიანიჭა TypeScript-მა `city` -ს?

`string`

2. `temperature` -ს?

`number`

3. `isRaining` -ს?

`boolean`

შემდეგ ჰკითხე:

```
temperature = "hot";
```

სწორია თუ არა? რატომ?

არა — TypeScript-მა `temperature` უკვე `number` ტიპად განსაზღვრა.

სავარჯიშო 5 — შექმენი საკუთარი TypeScript პროგრამა

შექმენით ოთხი ცვლადი მანქანის შესახებ:

- მანქანის ბრენდი — `string`;
- გამოშვების წელი — `number`;
- სიჩქარე — `number`;
- მოძრაობს თუ არა — `boolean`.

მაგალითად:

```
const carBrand: string = "BMW";  
const carYear: number = 2020;  
  
let speed: number = 0;  
let isMoving: boolean = false;
```

შემდეგ მანქანამ დაიწყო მოძრაობა:

```
speed = 50;  
isMoving = true;
```

შემდეგ სიჩქარე გახდა:

```
speed = 80;
```

აქ უკვე ბავშვმა უნდა ახსნას, რატომ ვერ ჩავწერთ:

```
speed = "fast";
```